

Team 9 (Section G)

Authors: Shikshith V (PES1UG24AM400), Sateesh E (PES1UG24AM397), Greeshma K (PES1UG24AM368), Rushikesh Praveen Patil (PES1UG24AM394)

Date: 02-10-2026

Architecture: Layered Modular Monolith + REST API + Background Scheduler Worker

This document picks up where the Software Requirements Specification (SRS) v1.0 left off. Requirement IDs (PWM-F-###, PWM-NF-###, PRJ-SR-###) are copied exactly as they appear in the SRS Requirements Traceability Matrix.

Role	Member	Architecture Responsibility
Frontend Lead	Shikshith V	React UI and client-side architecture
Backend Lead	Sateesh E	REST API and business/service layer
Database Lead	Greeshma K	PostgreSQL schema and persistence
DevOps & Security Lead	Rushikesh Praveen Patil	External integrations, scheduler, security and deployment

Document Control

Version	Date	Author	Change Summary
1.0	28-09-2026	Team 9	Initial SAD based on the supplied SAD template and the Personal Wealth Management SRS.
1.1	29-09-2026	Team 9	Design review fixes: corrected the component and deployment views; added the data model, scheduler design, and the valuation and alert rules; added the logout, password-reset, lockout, throttling and stale-price designs; added encryption at rest, the administrator role and the full API set; traceability now uses the requirement IDs.
1.2	29-09-2026	Team 9	SRS alignment corrections only: clarified individual user portfolio isolation; added numbered Administrator requirements; explicitly traced the alert baseline.
1.3	02-10-2026	Team 9	Final clean-up: put back the document-control fields and the approvals heading; removed the duplicated scope text and the duplicate PWM-F-040 traceability row; restored the openissue notes on administrator requirements, requirement-ID consistency and email verification; matched the formatting of the rest of the SAD.

Approvals

Role	Name	Signature / Date
Team Lead	Shikshith V	
Course Coordinator		

Table of Contents

1. Introduction (purpose, scope, audience, definitions)

2. Document Overview

3. Architecture (goals, stakeholders, components, pattern, technology, risks, traceability, security, quality tactics)
4. Design (overview, sequences, API, data model, valuation and alert rules, scheduler, error handling, UX, deployment, open issues)
5. Appendices (glossary, references, tools, ADR, implementation checklist)

1. Introduction

1.1 Purpose

This document describes the architecture and design of the Personal Wealth Management Software. It turns the functional, non-functional, security and acceptance requirements in the SRS (v1.0) into a concrete software structure. We chose a Layered Modular Monolith exposed through REST APIs, plus a separate background worker process that handles periodic valuation, alert evaluation and notification delivery.

1.2 Scope

The system manages personal portfolios for individual registered users. Each user's portfolio is kept separate from everyone else's and holds real estate, gold, stocks/equity and cash/bank balances in INR. The architecture covers:

- A frontend web application for managing and visualizing the portfolio.
- A REST API for authentication, asset management, dashboard, reports, valuations, alerts, notifications and administration.
- Relational persistence for users, assets, valuations, the price cache, alert configuration and audit records.
- A background scheduler worker for market-price refresh, alert evaluation and notification delivery.
- Integration with stock/gold data providers and an email provider.

Out of scope, as in the SRS: real money transactions, brokerage/trading execution, tax filing, sharing portfolios between users or family/shared accounts, live banking APIs and multi-currency support.

1.3 Audience

Developers, QA engineers, security reviewers, instructors, maintainers and project stakeholders.

1.4 Definitions

Term	Definition
SAD	Software Architecture and Design Specification
SRS	Software Requirements Specification
REST	Representational State Transfer
API	Application Programming Interface
JWT	JSON Web Token
TLS	Transport Layer Security
CRUD	Create, Read, Update, Delete
RBAC	Role-Based Access Control
ADR	Architecture Decision Record
RTM	Requirements Traceability Matrix (SRS Section 8)

2. Document Overview

2.1 How to use this document

Use this document as the architecture baseline before and during implementation. Section 3 covers the architecture itself: components, technology choices, risks, requirement traceability, security and quality tactics. Section 4 goes into the detailed design: sequence flows, API, data model, valuation and alert rules, scheduler, error handling, UX and deployment. Section 5 has the supporting material.

2.2 Related Documents

Document	Relationship
Software Requirements Specification (SRS) v1.0	The requirements this architecture has to meet. Its Section 8 holds the Requirements Traceability Matrix (RTM) and the test case IDs.
Software Test Plan (STP)	To be written once this design baseline is approved, using the acceptance criteria and test case IDs already defined in the SRS. Test writing can run in parallel with implementation.
Project source code / API specification	Implementation artifacts that must conform to this design.

3. Architecture

3.1 Goals & Constraints

Architecture Goals

- Keep presentation, API, business logic, persistence and integration as separate responsibilities.
- Stay modular without taking on the complexity of distributed services.
- Provide secure access to user-specific financial data (SRS 5.1).
- Run periodic valuation, alerting and notification delivery reliably, outside the web request path.
- Deal with external API failures using a limited number of retries, cached prices and a clear stale indicator.
- Keep the system easy to test and maintain, and big enough for the SRS scale target of 1,000 users.

Constraints

- The external stock and gold data providers may rate-limit us or go down.
- Real-estate values and cash/bank balances are entered by hand; live banking integration is out of scope.
- Single currency (INR) in this version.
- The SRS asks for PostgreSQL 14+ and allows Node.js 18+ or Python 3.10+. This SAD goes with Node.js + Express.
- Deployment uses Docker on Linux.
- Market prices follow Indian exchange trading days and hours (IST).

3.2 Stakeholders & Concerns

Stakeholder	Architecture Concerns
Portfolio User	Correct calculations, privacy, usability, availability, secure access.
Administrator	Price-feed configuration, system health visibility, account-level support (unlock, disable) without access to portfolio data.
Development Team	Modularity, clear interfaces, testability, maintainability.
QA / Evaluators	Requirement traceability, predictable behavior, testable components.
System Maintainer	Deployment simplicity, logs, monitoring, external API configuration, backups.
Security Reviewer	Authentication, authorization, TLS, password protection, data isolation, encryption at rest.

3.3 Component (UML) Diagram

The diagram shows the Layered Modular Monolith we selected. The backend is a single codebase that runs as two processes: the API process (controllers, services, repositories, adapters) and the scheduler worker. Both call the same service layer, so the valuation and alert logic lives in one place only. Controllers never reach repositories or adapters directly, and only repositories talk to PostgreSQL.

3.4 Component Descriptions

Component	Responsibility	Main Dependencies
-----------	----------------	-------------------

React Frontend (DashboardUI)	Dashboard, forms, charts, reports, alert configuration, notification bell, admin console. Runs in the browser; served as a static build by the reverse proxy.	REST API
Reverse Proxy	TLS 1.2+ termination, HTTP refusal, static file serving, forwards /api/* to the API process, security headers.	API process
REST Controllers + Middleware	HTTP endpoints, schema validation, authentication, RBAC, rate limiting, error mapping. No business logic.	Service Layer
AuthModule	Registration, login, logout, JWT issue/verify, token revocation, password reset, lockout, ownership checks.	User Repository, Token Repository, NotificationService
AssetModule	CRUD for real estate, gold, stock and cash records; input validation; ticker lookup.	Asset Repository, Instrument Repository, ValuationService
ValuationService	Calculates current and historical values, applies market prices, records manual valuations, maintains price cache, current_value and net-worth snapshots.	Repositories, Market Data Adapters
DashboardModule	Net worth, allocation and trend data.	ValuationService, Snapshot Repository
ReportModule	CSV and PDF generation of assets and valuation history.	Asset and Valuation Repositories
AlertService	Alert rule management, threshold evaluation, 24hour throttling.	Repositories, NotificationService
NotificationService	Writes notifications to the outbox (email and inapp); delivery is done by the worker with retry.	Notification Repository, Notification Adapter
AdminModule	Price-feed settings, system health, global refresh, unlock and disable accounts. No access to portfolio data.	Settings, Job, User Repositories
Scheduler Worker	Runs scheduled jobs (Section 4.6) by calling the same services as the API. Separate process, same codebase.	ValuationService, AlertService, NotificationService
Market Data Adapters	Isolate provider-specific stock and gold API calls; retry with backoff; normalize responses.	External APIs
Notification Adapter	Sends email through the provider (SMTP or API).	Email provider
Repositories	Abstract PostgreSQL persistence; every query takes the calling user id.	PostgreSQL

3.5 Chosen Architecture Pattern and Rationale

Chosen architecture: Layered Modular Monolith + REST API + Background Scheduler Worker.

The backend is one codebase and one deployable image, not a set of separately deployed services.

Inside it, modules are split by responsibility and arranged in layers. The same image starts in two modes, as the API process and as the worker process. We need the worker because valuation refresh, alert checks and email delivery cannot depend on someone having a browser request open, and because a slow provider should never hold up the API.

Layer / Element	Role
Presentation Layer	React web interface (browser).
REST API Layer	HTTP/JSON interface, middleware chain (rate limit, authentication, RBAC, validation).
Application / Service Layer	Business workflows: authentication, assets, valuation, dashboard, reports, alerts, notifications, administration.
Repository / Data Access Layer	Parameterized, user-scoped access to PostgreSQL.
Persistence Layer	PostgreSQL storage.
Integration Layer	Stock/gold providers and email provider behind adapters.
Scheduler Worker	Periodic jobs calling the service layer.

A modular monolith suits the size of this project and the simple deployment target. REST keeps the contract between frontend and backend explicit and easy to test. Microservices would add deployment and inter-service communication overhead that a 1,000-user target does not justify. We keep the module boundaries clean, so a module such as valuation could be pulled out later if needed.

3.6 Technology Stack & Data Stores

Area	Selected Technology	Purpose
Frontend	React	Responsive web application.
Backend	Node.js 18+ + Express	REST API and service execution.
Validation	Schema validation library (for example zod or Joi)	Request validation at the controller.
Communication	REST / JSON over HTTPS	Frontend-backend communication.
Database	PostgreSQL 14+	Relational persistence (NUMERIC for money).
Authentication	JWT (httpOnly cookie) + argon2id password hashing + revocation table	Authentication, logout invalidation, protected API access.
Scheduler	node-cron in a separate worker process, PostgreSQL advisory locks	Periodic valuation, alert, delivery and cleanup jobs.
External Data	Stock API + Gold API	Market-linked prices.
Notifications	Email provider (SMTP or API) + in-app notifications table	Alert delivery and password-reset email.
PDF / CSV	PDF generation library (for example pdftkit), streaming CSV writer	Reports.

Deployment	Docker Compose on Linux, nginx reverse proxy	Containerized runtime, TLS termination.
CI/CD	GitHub Actions	Lint, tests, image build and deploy.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Market API downtime	Current valuation unavailable.	Retry with backoff, keep last successful price, mark data stale (PWM-F-024).
API rate limiting	Refresh jobs delayed or rejected.	One request per distinct symbol per cycle, batching, configurable schedule, bounded concurrency.
Incorrect or delayed provider price	Wrong valuation.	Store price_as_of and source; reject nonpositive or wildly deviating prices (for example more than 50% change in one cycle) and flag for admin review.
Incorrect user input	Incorrect portfolio totals.	Client and server validation and database CHECK constraints.
Floating-point rounding	Totals differ from sum of parts.	NUMERIC in PostgreSQL, decimal arithmetic in code, money serialized as decimal strings.
Cross-user data access	Privacy/security violation.	JWT authentication, ownership checks in every service call, user_id-scoped repositories, UUID identifiers.
Stolen or leaked token	Account takeover.	httpOnly Secure SameSite cookie, expiry, revocation on logout and password reset, token_valid_after.
Scheduler failure or duplicate run	Stale valuations, or double alerts.	Advisory locks, job_runs history, health endpoint, catch-up after restart, manual refresh endpoints.
Notification failure	User may not receive alert.	Outbox with retry and backoff, delivery status kept, in-app copy always stored.
Database failure or data loss	Application data unavailable or lost.	Health checks, daily encrypted backups, tested restore.
Secret leakage (JWT secret, API keys)	Compromise of sessions or provider quota.	Secrets only in environment/Docker secrets, never in git or logs; rotation procedure.

3.8 Traceability to Requirements

Module names match the SRS RTM (AuthModule, AssetModule, ValuationService, DashboardUI, ReportModule, AlertService). The last column points to sections of this document.

Req ID	Design element	Section
PWM-F-001	AuthModule: POST /api/auth/register, password policy, argon2id hash, users table.	4.3, 4.4
PWM-F-002	AuthModule: POST /api/auth/login, JWT (sub, jti, iat, exp) in httpOnly cookie.	4.2.1

PWM-F-003	AuthModule: /api/auth/forgot-password and /reset-password, password_reset_tokens (30 min, single use), email through outbox.	4.2.3
PWM-F-010	AssetModule: POST /api/assets (REAL_ESTATE), real_estate_details.	4.3, 4.4
PWM-F-011	AssetModule: POST /api/assets (GOLD), gold_details with weight and purity validation.	4.3, 4.4

Req ID	Design element	Section
PWM-F-012	AssetModule: POST /api/assets (STOCK), GET /api/market/symbols, instruments table, stock_details.	4.2.5, 4.3
PWM-F-013	AssetModule: POST /api/assets (CASH), cash_details.	4.3, 4.4
PWM-F-014	AssetModule: PUT/DELETE /api/assets/:id; soft delete; current_value and today's snapshot updated in the same transaction.	4.3, 4.4
PWM-F-015	Controller schema validation (HTTP 400) plus database CHECK constraints (values at least 0).	4.3, 4.4
PWM-F-020	ValuationService + worker stock-refresh job; schedule in system_settings (default once per trading day).	4.5, 4.6
PWM-F-021	ValuationService + worker gold-refresh job; gold rate per purity in price_cache.	4.5, 4.6
PWM-F-022	PUT /api/assets/:id/valuation writes a valuations row with source MANUAL and timestamp.	4.3, 4.5
PWM-F-023	valuations table indexed on (asset_id, valued_at); GET /api/valuations/history with from/to.	4.3, 4.4
PWM-F-024	Market Adapter retry; price_cache.last_status and price_as_of; isStale and priceAsOf in API responses.	4.2.4, 4.5
PWM-F-030	DashboardModule: GET /api/dashboard, total = sum of current_value of non-deleted assets.	4.3, 4.5
PWM-F-031	GET /api/dashboard allocation; percentages adjusted to sum to exactly 100.00.	4.5
PWM-F-032	GET /api/dashboard/trend?range=1M 6M 1Y ALL from net_worth_snapshots.	4.3, 4.4
PWM-F-033	ReportModule: GET /api/reports/export/csv and /pdf (assets and valuation history).	4.3
PWM-F-040	AlertService: /api/alerts CRUD, alert_rules; applied at the next valuation cycle. Percentage change uses the latest valuation recorded at least 24 hours earlier; net-worth uses the previous day's snapshot.	4.3, 4.5
PWM-F-041	Notification outbox (email and in-app), delivery job every minute; normal-condition notification delivery is designed to meet the SRS acceptance target of within 5 minutes of breach detection.	4.6
PWM-F-042	alert_rules.last_triggered_at; 24-hour throttle check before enqueueing.	4.5
PWM-NF-001	Cached current_value, daily snapshots, indexes, no external call on the request path.	3.10

PWM-NF-002	One fetch per distinct symbol, batching, bounded concurrency, timeouts.	3.10, 4.6
PWM-NF-003	Restart policies, health checks, backups, uptime monitoring.	3.10, 4.9
PWM-NF-004	Responsive React layout from 360 px.	3.10, 4.8
PWM-NF-005	user_id-scoped schema and indexes; capacity estimate.	3.10
PRJ-SR-001	TLS 1.2+ at proxy, HTTPS-only adapters, plain HTTP refused.	3.9
Req ID	Design element	Section
PRJ-SR-002	argon2id salted hash.	3.9
PRJ-SR-003	JWT expiry (default 24 h), revoked_tokens, token_valid_after.	3.9, 4.2.2
PRJ-SR-004	Ownership checks, HTTP 403, UUID ids, user_id-scoped repositories.	3.9
PRJ-SR-005	failed_login_attempts, locked_until, audit_log.	3.9, 4.2.1
SRS Section 2.3	AdminModule: GET /api/admin/health exposes system health, scheduler status and recent job results.	4.3, 4.6
SRS Section 2.3	AdminModule: GET/PUT /api/admin/settings manages market-data refresh schedules and feed settings.	4.3, 4.6
SRS Section 2.3	AdminModule: POST /api/admin/users/:id/unlock and /disable manage account support actions; actions are audited.	3.9, 4.3
SRS Section 2.3	AdminModule: administrator access is restricted from user portfolio data.	3.9, 4.3

The Administrator role comes from SRS Section 2.3. We keep it as an architectural role for system maintenance, price-feed configuration, health monitoring and account support. Since the SRS gives it no functional requirement IDs, and we did not want to make any up, its capabilities are traced to SRS Section 2.3 instead of the functional RTM.

3.9 Security Architecture

Security is applied in layers: transport, authentication, authorization, validation and persistence. Every request to the API process goes through the same fixed path, and each step can be tested:

Browser --HTTPS/TLS 1.2+--> Reverse proxy --HTTP (internal network)--> API process

API middleware chain (in order):

- 1. rate limit (per IP; stricter on /api/auth/*)**
- 2. Origin check on state-changing requests**
- 3. read auth cookie, verify JWT signature and exp**
- 4. load user; reject if jti revoked, user disabled, or iat < token_valid_after**
- 5. role check (USER / ADMIN)**
- 6. schema validation of body, query and params**
- 7. controller -> service: ownership check on the target resource**
- 8. repository: parameterized SQL, always filtered by user_id**

Area	Design	Requirement
-------------	---------------	--------------------

Transport	TLS 1.2+ ends at the reverse proxy. Only port 443 is open and plain HTTP is not served. HSTS is on. Adapters call providers over HTTPS with certificate validation enabled. If the database ever moves to another host, PostgreSQL connections must use TLS.	PRJ-SR-001
Password storage	argon2id with OWASP-recommended parameters and a salt for each password. Nothing is stored in plaintext or in a reversible form. Password policy: at least 10 characters, letters and digits, not on a common-password list, maximum 128.	PRJ-SR-002, PWM-F-001
Session tokens	The JWT is signed with a secret of at least 256 bits taken from the environment. Claims: sub, jti, iat, exp (default 24 h, configurable). It sits in an httpOnly, Secure, SameSite=Strict cookie, so scripts cannot read it.	PRJ-SR-003

Area	Design	Requirement
Logout and revocation	On logout the jti goes into revoked_tokens until it expires, and the middleware rejects any revoked jti. A password reset sets users.token_valid_after, which kills all older tokens. A cleanup job removes expired rows.	PRJ-SR-003
Authorization	Authentication middleware covers every route except register, login, forgot/reset password and health. Each service call gets the user id from the token, never from the request body, and repositories always filter by user_id. If a resource exists but belongs to another user, the API returns 403, as the SRS requires. IDs are random UUIDs, so they cannot be guessed.	PRJ-SR-004
Lockout	After 5 failed logins in a row, locked_until is set to now + 15 minutes. The lock is checked before the password, the event goes to audit_log, and a successful login resets the counter. An unknown email and a wrong password get identical responses.	PRJ-SR-005
Administrator role	users.role is USER or ADMIN. Admin routes live under /api/admin and need the ADMIN role. Admins can change settings, view health, unlock or disable accounts and trigger a global refresh, but they cannot read any user's assets or valuations. Admin accounts are created by a deployment script, never through public registration, and every admin action is audited.	SRS Section 2.3
Input handling	Every controller validates the schema, SQL is always parameterized, and CHECK constraints in the database act as a second barrier. React escapes output and nothing is rendered as raw HTML. Content-Security-Policy and other headers are set through helmet. CSV export neutralizes cells that start with =, +, - or @.	PWM-F-015
Cross-site protection	SameSite=Strict cookie plus an Origin check on state-changing requests. The API is same-origin through the proxy, and CORS allows no other origin.	PRJ-SR-004
Data at rest	The PostgreSQL data volume and the backups sit on encrypted storage (LUKS or provider volume encryption). Backups are encrypted again before they leave the host. We do not store card numbers, bank credentials or government IDs.	SRS 5.1.1

Secrets	The JWT secret, provider API keys and SMTP credentials come in as environment variables or Docker secrets and are never committed. The rotation procedure is in the runbook; rotating the JWT secret logs everyone out.	SRS 5.1.1
Logging and audit	Structured logs with a request id. Passwords, tokens, reset links, API keys and full request bodies are never logged. audit_log records login success and failure, lockout, logout, password reset and admin actions.	PRJ-SR-005
Platform hardening	Containers run as non-root, images and dependencies are scanned in CI, and the database is not reachable from outside the Docker network.	SRS 5.1.1

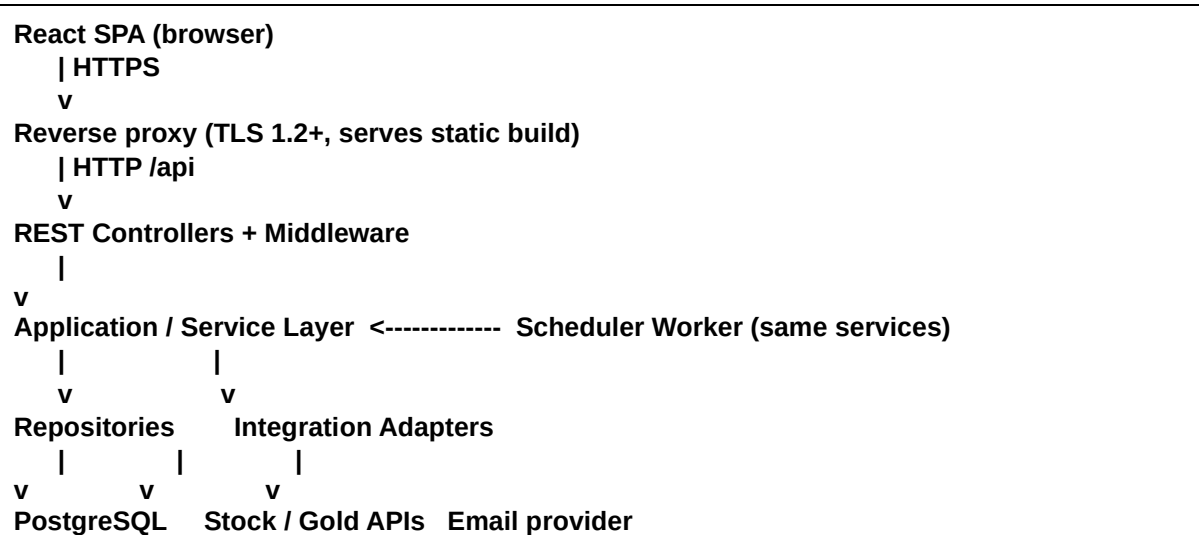
3.10 Quality Attribute Design

Requirement	Design tactics
PWM-NF-001: dashboard p90 within 2 s	The dashboard reads the cached assets.current_value, so nothing is recalculated on each request. The trend reads net_worth_snapshots (one row per user per day) instead of the full valuation history. Indexes: assets(user_id, deleted_at) and valuations(asset_id, valued_at DESC). No external API call happens on the request path. We also use gzip, hashed static assets with long cache headers, and route-level code splitting in React.
PWM-NF-002: 500 assets refreshed within 5 minutes	Each distinct symbol or purity is fetched once per cycle, no matter how many users hold it. We batch requests where the provider allows it, limit concurrency (for example 5), use a 10 s timeout per call and bulk-insert valuations. Duration is recorded in job_runs.
PWM-NF-003: 99.5% monthly uptime	99.5% still allows about 3.6 hours of unplanned downtime a month. Containers use restart: unless-stopped with health checks, the API is stateless, PostgreSQL has daily encrypted backups with a tested restore, an external uptime monitor watches /api/health, and maintenance windows are announced in advance.
PWM-NF-004: usable from 360 px	Mobile-first CSS with breakpoints at 360, 768 and 1280 px. Tables and charts scroll inside their own container, and forms switch to a single column on narrow screens.
PWM-NF-005: 1,000 users without redesign	Every table is keyed by user_id. Rough estimate: 1,000 users x about 20 assets x 365 daily valuation rows gives about 7.3 million valuation rows a year. One PostgreSQL instance handles that comfortably with the (asset_id, valued_at) index, and we can partition valuations by year if it grows. The API is stateless and the job locks make extra instances safe.

4. Design

4.1 Design Overview

The frontend talks only to the REST API. Controllers turn HTTP requests into service calls. Services hold the business rules and use repositories for persistence and adapters for external providers. The scheduler worker calls the same services, so valuation and alert logic is never duplicated. All outgoing email, including password-reset email, goes through the notification outbox and is sent by the worker.



4.2 UML Sequence Diagrams

4.2.1 User Login (with logout)

4.2.2 Token Validation and Logout

4.2.3 Password Reset

4.2.4 Scheduled Valuation Refresh and Alert Evaluation

Gold follows the same flow (rate per purity instead of price per symbol), and so do the manual refresh endpoints.

4.2.5 Add a Stock Holding

4.3 API Design

4.3.1 Conventions

- Base path is /api, JSON only, UTF-8. Authentication uses the auth cookie, and state-changing requests must come from the same origin.
- Identifiers are UUIDs. Timestamps are ISO 8601 in UTC.
- Money, quantity and weight are sent as decimal strings (for example "2800.00") to avoid floating-point errors. Currency is INR.
- List endpoints take page and limit (default 1 and 50, maximum 200) and return items, page, limit and total.
- Every market-priced value carries priceAsOf and isStale. isStale is true when the latest refresh attempt for that price failed.
- Errors use one shape: {"error": {"code": "VALIDATION_ERROR", "message": "...", "fields": [{"field": "quantity", "message": "must be 0 or more"}]}}.

4.3.2 Endpoints

Method	Endpoint	Purpose	Auth
POST	/api/auth/register	Create user account.	Public
POST	/api/auth/login	Authenticate; sets the auth cookie.	Public
POST	/api/auth/logout	Revoke the current token; clears the cookie.	JWT
POST	/api/auth/forgot-password	Start reset; always returns 202.	Public
POST	/api/auth/reset-password	Complete reset with token and new password.	Public
GET	/api/auth/me	Current user id, email and role.	JWT
GET	/api/assets	List own assets; filters type, page, limit.	JWT
POST	/api/assets	Create asset; assetType selects the fields.	JWT
GET	/api/assets/:id	Read one owned asset.	JWT
PUT	/api/assets/:id	Update owned asset.	JWT
DELETE	/api/assets/:id	Soft-delete owned asset.	JWT
PUT	/api/assets/:id/valuation	Manual valuation (real estate, cash); writes a MANUAL valuation.	JWT
GET	/api/market/symbols	Ticker lookup; query parameter q.	JWT
GET	/api/dashboard	Net worth, allocation, stale flag.	JWT
GET	/api/dashboard/trend	Net-worth series; range = 1M, 6M, 1Y or ALL.	JWT
GET	/api/valuations/history	Valuation history; assetId, from, to.	JWT
POST	/api/valuations/refresh	Refresh the caller's holdings only; 1 per 5 minutes per user; 429 if exceeded.	JWT
GET	/api/reports/export/csv	CSV of assets and valuation history.	JWT
GET	/api/reports/export/pdf	PDF of assets and valuation history.	JWT
GET	/api/alerts	List own alert rules.	JWT
POST	/api/alerts	Create alert rule.	JWT
PUT	/api/alerts/:id	Update alert rule.	JWT
DELETE	/api/alerts/:id	Delete alert rule.	JWT
GET	/api/alerts/events	History of triggered alerts.	JWT
GET	/api/notifications	List in-app notifications; unread filter.	JWT

PATCH	/api/notifications/:id/read	Mark a notification read.	JWT
Method	Endpoint	Purpose	Auth
GET	/api/admin/settings	Read refresh schedules and feed settings.	ADMIN
PUT	/api/admin/settings	Update refresh schedules and feed settings.	ADMIN
GET	/api/admin/health	Job status, last success, feed failures, queue depth.	ADMIN
POST	/api/admin/valuations/refresh	Global refresh of all holdings.	ADMIN
GET	/api/admin/users	Account list with status; no portfolio data.	ADMIN
POST	/api/admin/users/:id/unlock	Clear a lockout.	ADMIN
POST	/api/admin/users/:id/disable	Disable an account.	ADMIN
GET	/api/health	Liveness and database readiness; no details.	Public

4.3.3 Example Contracts

POST /api/assets. The assetType field selects the required fields:

```

STOCK
{"assetType":"STOCK","ticker":"RELIANCE","exchange":"NSE",
 "quantity":"10","averageBuyPrice":"2800.00"}
GOLD
{"assetType":"GOLD","form":"COIN","purityKarat":22,
 "weightGrams":"20.000","purchasePrice":"110000.00"}
REAL_ESTATE
{"assetType":"REAL_ESTATE","name":"Flat, Adyar","location":"Chennai",
 "purchaseValue":"6500000.00","purchaseDate":"2021-03-15",
 "currentEstimatedValue":"8200000.00"}
CASH
{"assetType":"CASH","accountLabel":"Savings account","balance":"150000.00"}

```

Response 201 (stock example):

```

{"id":"5f0c9e52-8b7d-4a1e-9c3f-2d6a41b7e001","assetType":"STOCK",
 "ticker":"RELIANCE","exchange":"NSE","quantity":"10",
 "averageBuyPrice":"2800.00","currentValue":"28500.00", "priceAsOf":"2026-09-
28T10:30:00Z","isStale":false}

```

GET /api/dashboard response:

```

{"netWorth":"10345000.00","currency":"INR","asOf":"2026-09-29T04:30:00Z",
 "hasStaleData":false,
 "allocation":[
 {"assetType":"REAL_ESTATE","value":"8200000.00","percent":"79.26"},
 {"assetType":"STOCK","value":"1200000.00","percent":"11.60"},
 {"assetType":"GOLD","value":"645000.00","percent":"6.23"},
 {"assetType":"CASH","value":"300000.00","percent":"2.91"}]}

```

4.3.4 Status Codes

Code	Use
200 / 201 / 204	Success; created; success with no body (logout, delete).
202	Accepted; used for forgot-password so the reply never reveals whether an email exists.
Code	Use
400	Malformed request or validation failure (wrong type, negative value, missing field).
401	Missing, expired, revoked or invalid token; wrong login credentials.
403	Authenticated but not allowed: another user's resource, or missing ADMIN role.
404	Resource does not exist.
409	Conflict, for example email already registered.
422	Well-formed but rejected by a business rule, for example unknown ticker.
423	Account locked after 5 failed logins; response includes the retry time.
429	Rate limit exceeded; Retry-After header set.
500 / 503	Unexpected server error / dependency unavailable; safe message only.

4.4 Data Model

The schema is relational. Fields common to all assets live in assets, and each asset class has its own detail table so it can have its own constraints. Money is NUMERIC(18,2), quantities NUMERIC(18,4) and weights NUMERIC(12,3); we do not use floating-point types. All timestamps are TIMESTAMPTZ.

4.4.1 Tables

Table	Key columns	Notes
users	id, email (unique, lowercase), password_hash, role, failed_login_attempts, locked_until, token_valid_after, disabled_at, created_at	Lockout and sessioninvalidation state lives here.
assets	id, user_id, asset_type, name, current_value, currency, created_at, updated_at, deleted_at	current_value is the cached latest valuation. Soft delete keeps historical snapshots explainable, and deleted rows are left out of totals and exports.
real_estate_details	asset_id, location, purchase_value, purchase_date, estimated_value_updated_at	CHECK values at least 0.
gold_details	asset_id, form (JEWELLERY, COIN, BAR), purity_karat, weight_grams, purchase_price	CHECK weight and price at least 0, purity 1 to 24.

stock_details	asset_id, symbol, exchange, quantity, average_buy_price	Symbol references instruments. CHECK values at least 0.
cash_details	asset_id, account_label, balance	Manual balance.
instruments	symbol, exchange (primary key), name, active, updated_at	Symbol master used to validate tickers; refreshed weekly.
price_cache	instrument_key (STOCK:NSE:RELIANCE, GOLD:22K), price, price_as_of, last_attempt_at, last_status, source	Source of truth for the stale flag.
Table	Key columns	Notes
valuations	id, asset_id, value, unit_price, source (MARKET or MANUAL), valued_at	Index (asset_id, valued_at DESC). Supports PWMF-023.
net_worth_snapshots	user_id, snapshot_date, total, breakdown (JSONB)	One row per user per day; upserted on refresh and on asset changes.
alert_rules	id, user_id, scope (NET_WORTH or ASSET), asset_id, threshold_percent, direction, channel, active, last_triggered_at	last_triggered_at drives the 24-hour throttle.
alert_events	id, rule_id, baseline_value, current_value, change_percent, triggered_at	History shown to the user.
notifications	id, user_id, type, subject, body, channel, status, attempts, next_attempt_at, sent_at, read_at, created_at	Outbox and in-app inbox in one table.
password_reset_tokens	id, user_id, token_hash, expires_at, used_at, created_at	Only the SHA-256 hash is stored; 30-minute expiry; single use.
revoked_tokens	jti, user_id, expires_at	Denylist for logged-out tokens; purged after expiry.
audit_log	id, user_id (nullable), event_type, ip, user_agent, details (JSONB), created_at	No secrets in details.
job_runs	id, job_name, started_at, finished_at, status, items_processed, error_summary	Scheduler history for health checks.
system_settings	key, value (JSONB), updated_by, updated_at	Refresh schedules, token lifetime, feed configuration (no secrets).

4.5 Valuation and Alert Rules

- Stock: quantity x latest price for the symbol. Profit/loss is shown against quantity x average buy price.
- Gold: weight in grams x rate per gram for the purity. If the provider only publishes 24-karat, other purities use $\text{rate}_{24} \times \text{karat} / 24$. Making charges and wastage are not included, and the UI says so.

- Real estate and cash: the value the user entered. PUT /api/assets/:id/valuation stores it with a timestamp (PWM-F-022), and any manual change to a value writes a MANUAL row to valuations.
- Net worth: the sum of current_value over the user's non-deleted assets (PWM-F-030). Editing or deleting an asset updates current_value and today's snapshot in the same transaction, so totals change immediately (PWM-F-014).
- Allocation: each class value divided by net worth, rounded to 2 decimals. The rounding remainder is added to the largest slice so the total is exactly 100.00 (PWM-F-031).
- Stale price: a market price counts as stale when the latest refresh attempt failed even after retries. The cached price stays in use and the API returns isStale = true with priceAsOf (PWMF-024). The next successful refresh clears the flag.
- Alert baseline: change is measured against the latest valuation recorded at least 24 hours earlier (for net worth, the previous day's snapshot). $\text{change_percent} = (\text{current} - \text{baseline}) / \text{baseline} \times 100$. An alert fires when its absolute value reaches the threshold (direction defaults to both). If there is no baseline, or the baseline is zero, no alert is raised.
- Throttling: if alert_rules.last_triggered_at is less than 24 hours old, no new event or notification is created for that rule (PWM-F-042).

4.6 Scheduler and Background Jobs

Jobs run in the worker container. The schedules below are defaults; an administrator can change them through system_settings (PWM-F-020 configurable interval). Times are IST.

Job	Default schedule	What it does
stock-refresh	16:00, Monday to Friday, trading days only	Fetch price per distinct held symbol, update price_cache, revalue holdings, write valuations and snapshots, then run alert evaluation.
gold-refresh	Daily 10:00	Fetch rate per purity in use, revalue gold, write valuations and snapshots, then run alert evaluation.
notification-delivery	Every minute	Send PENDING notifications, with up to 5 attempts at delays of 1, 2, 4 and 8 minutes; the status is kept. Because delivery runs every minute, notifications normally go out well within the SRS target of 5 minutes from breach detection.
symbol-master-sync	Weekly	Refresh the instruments table.
cleanup	Daily	Delete expired revoked_tokens and password_reset_tokens, and job_runs older than 90 days.

- No double runs: each job takes a PostgreSQL advisory lock (pg_try_advisory_lock), so a second or restarted worker skips a job that is already running.
- Observability: every run writes job_runs (status, items, error). /api/admin/health shows the last success per job, and a job with no success for twice its interval is flagged.
- Catch-up: when the worker starts, a job whose scheduled time was missed in the last 6 hours runs once.
- Manual refresh: POST /api/valuations/refresh (own holdings only, rate limited, answered from the price cache if the cached price is under 5 minutes old) and POST /api/admin/valuations/refresh (all holdings).

- Provider calls: 10 s timeout, 3 retries at 1, 2 and 4 seconds plus jitter, HTTPS only, one request per distinct symbol per cycle.

4.7 Error Handling, Logging & Monitoring

Condition	Handling
Validation failure	HTTP 400 with field-level messages.
Business-rule rejection	HTTP 422 (for example unknown ticker).
Unauthenticated request	HTTP 401; nothing from the protected resource is returned.
Unauthorized access	HTTP 403; cross-user access is rejected and audited.
Account locked / rate limited	HTTP 423 with retry time / HTTP 429 with Retry-After.
Condition	Handling
Missing resource	HTTP 404.
External market API failure	Retry with backoff; then cached price and stale indicator; failure counted for monitoring.
Email delivery failure	Retry with backoff; status FAILED after the last attempt; the inapp notification still exists.
Scheduler job failure	Logged in job_runs, retried at the next cycle, visible in admin health.
Unexpected server or database error	HTTP 500/503 with a safe message; details only in the internal log with the request id.

Logging: structured JSON logs with a request id. Passwords, tokens, reset links, API keys and full request bodies are never written. Audit events: login success and failure, lockout, logout, password reset request and completion, and admin actions.

Monitoring covers API error rate, dashboard endpoint latency, job status and duration, external API failures, notification failures, database health, disk space and backup success.

4.8 UX Design

The React interface is a responsive dashboard that works from 360 px up. Forms validate before submitting and show the server's field-level errors. Values that come from a failed refresh get a stale badge with the time of the last good price. A bell icon shows unread in-app notifications, and the lockout message tells the user when they can try again.

Login / Register / Forgot password / Reset password Dashboard +-- Total Net Worth (stale badge when needed) +-- Asset Allocation +-- Historical Trend (1M / 6M / 1Y / All) +-- Alerts and notifications (bell) Assets +-- Real Estate (add, edit, delete, manual revaluation) +-- Gold (add, edit, delete) +-- Stocks (add with ticker lookup, edit, delete) +-- Cash (add, edit, delete)
--

Reports (CSV, PDF)
Settings / Alert Thresholds
Admin (ADMIN role only)
 +-- Price-feed settings
 +-- System health
 +-- Accounts (unlock, disable)

4.9 Deployment Design

Container	Role	Notes
reverse-proxy (nginx)	TLS termination, serves the React build, forwards /api to the API.	Only container with a published port (443). Certificates from the chosen CA.
api	Express API process.	Stateless; non-root user; health check on /api/health.
Container	Role	Notes
worker	Scheduler and outbox delivery.	Same image as api, started in worker mode.
postgres	PostgreSQL 14+.	Named volume on encrypted storage; not published outside the Docker network.
backup	Daily pg_dump at 02:00, encrypted, rotated (default 14 days).	Copy kept off the host; restore tested before each release.

Configuration: DATABASE_URL, JWT_SECRET, STOCK_API_KEY, GOLD_API_KEY and the SMTP or email-provider credentials come from environment variables or Docker secrets and are never committed. CI/CD (GitHub Actions): lint, unit tests, integration tests against a PostgreSQL service container, dependency audit, image build and scan, then deploy to the Linux host with Docker Compose.

4.10 Open Issues & Next Steps

- SRS administrator requirements: Section 2.3 defines an Administrator role but gives no numbered functional requirement IDs. This SAD traces administrator functions to Section 2.3 rather than making up PWM-F IDs. If the SRS is revised, the RTM should be updated with the final IDs.
- Requirement-ID consistency: check that every reference uses the exact prefixes and numbering from the SRS (PWM-F-####, PWM-NF-#### and PRJ-SR-####). Any PWMF/PWM-F or PWM-SR/PRJ-SR mix-up should be fixed before implementation.
- Email verification: confirm whether registration needs email verification. The current SRS specifies registration but has no email-verification requirement or acceptance criterion, so this is still a product/SRS question.
- Choose the stock API provider, and decide how credentials and configuration are handled.
- Choose the gold-rate provider and check whether it gives every purity or only 24-karat (4.5 assumes we derive the rest).
- Choose the email provider.
- Decide how the exchange holiday calendar is kept up to date so the stock job skips non-trading days.

- Finalize the physical PostgreSQL schema (types, indexes, constraints) from Section 4.4 and freeze it as migrations.
- Confirm backup retention and the off-host backup location.
-
- Build the API, worker and authentication before any portfolio data is exposed.
- Write the Software Test Plan from the SRS test case IDs once this baseline is approved.

5. Appendices

5.1 Glossary

Term	Meaning
Layered Architecture	Organization into presentation, API, service, data-access and persistence layers.
Modular Monolith	One codebase and deployable image whose internal functions are separated into cohesive modules.
Scheduler Worker	Separate process that runs periodic jobs without a user request.
Stale Price	Cached market price in use because the latest refresh attempt failed.
Outbox	Table of pending notifications written by the services and delivered asynchronously by the worker.
Denylist (revoked_tokens)	Table of JWT ids that must be rejected even though they have not expired.
Advisory lock	PostgreSQL application-level lock used so a job never runs twice at once.
Soft delete	Marking a row deleted (deleted_at) instead of removing it.
argon2id	Memory-hard password hashing algorithm.
JWT	Signed token representing an authenticated session.
ADR	Architecture Decision Record documenting an important decision.

5.2 References

- Personal Wealth Management Software, Software Requirements Specification (SRS), Version 1.0, Team 9 (Section G), 07-09-2026, including its Section 8 Requirements Traceability Matrix.
- Software Architecture and Design Specification template supplied for the project.

5.3 Tools

Tool / Technology	Use
React	Frontend implementation.
Node.js + Express	REST API and worker.
node-cron, pg	Scheduling and PostgreSQL access.
PostgreSQL	Relational database.

Docker Compose + Linux, nginx	Deployment environment and reverse proxy.
GitHub + GitHub Actions	Version control, collaboration, CI/CD.
Jest and Supertest	Unit and API tests.
Postman	REST API testing.
Draw.io / PlantUML	UML and architecture diagrams.

5.4 Architecture Decision Record (ADR)

Decision	Rationale	Alternative rejected
Layered modular monolith, API and worker from one codebase	Simple deployment, one service layer for API and jobs, fits scale.	Microservices: extra complexity for 1,000 users.
REST / JSON over HTTPS	Explicit, testable contract.	GraphQL: not needed for this scope.
JWT in httpOnly cookie plus revoked_tokens table	Scripts cannot read the token; logout and reset can invalidate tokens.	Bearer token in localStorage: exposed to XSS. Stateless JWT only: cannot honor logout.
argon2id password hashing	Current best practice; meets PRJSR-002.	Plain SHA hashes: unsuitable.
Separate worker process with advisory locks	API stays responsive; safe if more than one instance runs.	Cron inside the API process: double runs when scaled.
Denormalized current_value and daily snapshots	Meets the 2 s dashboard target cheaply.	Recompute from history per request: slow at scale.
Class-table design for assets (common table plus detail tables)	Per-class constraints and clear validation.	One wide nullable table or unvalidated JSON.
NUMERIC money, decimal strings in JSON	No floating-point error in totals.	Float or integer paise everywhere: more conversion risk.
Soft delete for assets	Historical snapshots stay explainable.	Hard delete with cascade.
Notification outbox delivered by the worker	Alerts survive provider outages; retries are simple.	Send email inside the request or job: lost on failure.
HTTP 403 for another user's resource	Required by PRJ-SR-004; UUID ids prevent guessing.	404 for everything: does not match the SRS test.
Node.js + Express, React, PostgreSQL, Docker on Linux	Allowed by SRS 2.4; one language front to back.	Python backend: equally valid, not selected.

5.5 Architecture-to-Implementation Checklist

Order	Work Item	Owner
-------	-----------	-------

1	Freeze the database schema (Section 4.4) as migrations.	Database Lead
2	Set up repository, Docker Compose and the CI pipeline (lint, test, build).	DevOps
3	Implement authentication, authorization, lockout, audit log.	Backend + Security
4	Implement asset CRUD APIs with validation and ticker lookup.	Backend
5	Implement React shell, auth screens, dashboard skeleton and asset forms.	Frontend
6	Implement valuation services, market adapters and the price cache.	Backend + Database

Order	Work Item	Owner
7	Implement the scheduler worker, job locks and job_runs.	Backend + DevOps
8	Implement dashboard and trend endpoints, snapshots and charts.	Backend + Frontend
9	Implement alerts, the notification outbox and email delivery.	Backend + DevOps
10	Implement CSV and PDF reports.	Backend + Frontend
11	Implement the admin module and monitoring.	Backend + DevOps
12	Harden TLS and containers, set up encrypted backups, deploy on Linux.	DevOps
13	Write the Software Test Plan from the SRS test cases (start after baseline approval) and run it.	Entire Team